

School of Computing and Mathematics

ASSESSMENT BRIEF

Module Code and Title:	6G5Z0059 Software Design and Architecture
Assessment Set By:	Steve.Baker@mmu.ac.uk
Assessment ID:	CM014442
Assessment Weighting:	100%
Assessment Title:	1CWK100 Report
Version	1.0
Type:	Individual
Hand-In Deadline:	See Moodle
Hand-In Format and Mechanism:	Submission online via Moodle

Learning outcomes being assessed:

LO1: Design software systems using an industrial standard design process.

LO2: Identify and apply a range of software design patterns.

LO3: Evaluate software architecture and design.

Note: it is your responsibility to make sure that your work is complete and available for marking before the deadline. Make sure that you have followed the submission instructions carefully, and your work is submitted in the correct format, using the correct hand-in mechanism (e.g., Moodle upload). If submitting via Moodle, you are advised to check your work after upload, to make sure it has uploaded properly. If submitting via OneDrive, ensure that your tutors have access to the work. Do not alter your work after the deadline. You should make at least one full backup copy of your work.

Penalties for late submission

The timeliness of submissions is strictly monitored and enforced.

All coursework has a late submission window of 7 calendar days, but any work submitted within the late window will be capped at 40%, unless you have an agreed extension. Work submitted after the 7-day late window will be capped at zero unless you have an agreed extension. See the section on [Extensions](#) below for further information.

Please note that individual tutors are unable to grant any extensions to assessments.

Extensions

We understand that sometimes things happen that are beyond your control and that could have an impact on your assessments. Our [Assessment Extensions Procedure](#) is designed to help you continue your studies despite unexpected disruption which may impact your assessments. You can apply for an extension or defer your assessment to a later date directly in the module Moodle area.

Non-evidenced extension does not require you to submit evidence. It allows you to add a short extension (usually 2 days) to a deadline. This type of extension does not apply to event-based assessments such as in-class tests, exams, presentations, etc. and is not available for any reassessments. Non-evidenced extensions need to be added between a week before your assessment due date and the deadline; it isn't available to add after a deadline has passed.

Evidenced extension requires you to provide independent evidence of a situation which has impacted you. It allows you to apply for a longer extension than a non-evidenced extension and is also available for event-based assessment such as in-class tests, exams, presentations, etc.

Further information about extensions is available on the dedicated [Assessments page](#).

Personal Learning Plans

If you have a [Personal Learning Plan \(PLP\)](#) which states you can negotiate an extended deadline, submit an evidenced extension request on the Moodle site for the module.

Plagiarism

Plagiarism is the unacknowledged representation of another person's work, or use of their ideas, as one's own. Manchester Metropolitan University takes care to detect plagiarism, employs plagiarism detection software, and imposes severe penalties, as outlined in the [Student Code of Conduct](#) and [Academic integrity and misconduct](#). Poor referencing or submitting the wrong assignment may still be treated as plagiarism. If you are in doubt, seek advice from your tutor.

As part of a plagiarism check, you may be asked to attend a meeting with the Module Leader, or another member of the module delivery team, where you will be asked to explain your work (e.g. explain the code in a programming assignment). If you are called to one of these meetings, it is very important that you attend.

Use of generative AI

The use of generative AI is permitted in this assessment, so long as it is used in accordance with the instructions provided in the 'Are you allowed to use AI in assessments?' section of the [AI Literacy Rise Study Pack](#). All submitted work must be your own original content.

If you are unable to upload your work to Moodle

If you have problems submitting your work through Moodle, you can raise a ticket with the Assessment Management Team using the [Assist Portal](#). This must be done **before the published deadline**, else your work will be logged as a late submission. Alternatively, you can save your work into a single zip folder then upload the zip folder to your university OneDrive and submit a Word document to Moodle which includes a link to the folder. **It is your responsibility to make sure you share the OneDrive folder with the Module Leader, or it will not be possible to mark your work.**

Assessment Regulations

For full details of the rules and procedures that govern all assessed work to ensure students are fairly and objectively assessed see the [Undergraduate Assessment Regulations](#).

Formative Feedback:	<i>Lab sessions will be dedicated to assessment support and to provide verbal formative feedback. Additional support can be obtained in Tutor office hours (check Moodle for times).</i>
Summative Feedback:	<i>Written feedback will be provided after submission and before the individual feedback date shown on Moodle. This feedback will be written with reference to the summative marking scheme.</i>

Introduction to the Assessment Task

The assessment task is an opportunity to demonstrate your software design skills and knowledge of software design patterns and software architecture. The task is to design a small software product using the principles, patterns and processes covered in the taught materials (including the module textbook and background reading) and lab exercises of the Software Design and Architecture module.

The module is 100% coursework based, and this assessment component is weighted at 100% of the module marks. This is an individual activity.

Marks are awarded for meeting the functional and architectural requirements (outlined below) and for designing and writing the code in an object-oriented manner using common design patterns. There are additional marks available for implementing advanced features.

The deliverables (the things you need to produce and submit via Moodle) are:

- A Java **console** application that plays the game and meets some or all the functional specification below. A graphical user interface is **NOT** required.
- A **Q&A** document answering questions about your design.

The Submission

Your submission should be in the form of a ZIP file containing:

- The complete source code for your submission as an IntelliJ project.
- A Q&A file written in Markdown.

Please name your submission file as

Firstname_Lastname_StudentIDNumber.zip (Example: Steve_Williams_12345678.zip).

Moodle submission inboxes cannot accept files larger than 100 MB. It is unlikely that you will exceed this for this assessment. Should you need to submit files exceeding this size you are required to upload your project to your university-supplied OneDrive account and submit a link as part of the package you submit to Moodle (e.g., pasted into a text file). Prior to submission via Moodle, please ensure that you archive your project folders into a zip file and upload them to your OneDrive account.

The Assessment Project

You are simulating a prototype physical board game very loosely based on the Snakes and Ladders game. There is a demo in Lecture 1 that shows an example simulation in action. The game rules are discussed in the Appendix.

A simulation is a model of the board game which:

- Starts a game (setting up the board and player pieces)
- Takes an input of a roll of one die or two dice and automatically alternates between players.
- Tracks the position of the players around the board and implements the game rules.
- Outputs the move made by each player.

Your software is to be written as a console application (you do **NOT** need to create a GUI or Web interface).

Your software will need to output progress information and play statistics as play progresses.

The board game designers want to vary elements of the design to understand from the simulation how this affects game play, for example if it makes play too easy or too difficult.

You will also need to produce a Q&A document which evaluates your architecture and design.

Functional Specification

The simulation must follow the basic game rules (supplied in the Appendix).

The board game designers have asked for the simulation to output information to the console about the progress of a game:

1. When the game starts, output the size of the board, the number of players, the game rules being used (End, Hit and Wormhole) and the number of dice used for a simulation run. Example output which meets the functional specifications is shown in the Appendix.
2. Players take turns to play until the game is won.
3. Each turn there is a simulated dice roll. At each turn output the current player, the value of the dice roll(s) and the position of the player before and after the turn.
4. After each turn, output the running total of the number of turns taken by the current player.
5. At the end of the game (i.e. the game has been won) output the winning player.
6. At the end of the game (i.e. the game has been won) output the total number of turns taken by all the players.

A game simulation that follows the basic game rules and meets the functional specification is called the **basic game**. See the Appendix for a full explanation of the game rules with examples.

Functional Variations

The board game designers have asked for the simulation to be run with different variations.

Your program should automatically run games demonstrating each variation you have attempted.

Marks are awarded based on number of variations implemented.

- **Single Die:** Player throws a single 6-sided die each turn. Having implemented this variation, the simulation can run with either two dice or a single die.
- **Exact End: Player must land exactly on the END position to win:** The player needs to throw the exact number to reach END – if the number is too high, move to END and then back away, reversing back around the board (in effect bouncing back from the END position). Having implemented this variation, the simulation can run with two different rules for handling the END position.
- **Hit: Player stays where they are if they HIT another player:** If a player would **Hit** another player, the player stays where they are (the player's turn is forfeit). For example, if the **Red** player would have landed on a position occupied by the **Blue** player, the **Red** player stays where they are (as if the turn had not happened). Likewise, if a **Blue** player piece would have landed on a position occupied by **Red**, then the Blue player's turn is forfeit. Having implemented this variation, the simulation can run with two different rules for handling a HIT.
- **Teleport: Players can teleport through wormholes:** The board configuration may define pairs of positions that are connected via **wormholes**. A wormhole connects two positions on the board, if a player lands either end of a 'wormhole' they are 'teleported' to the other end. A wormhole can either advance a player or set them back depending on the position of the wormhole. Having implemented this variation, the game can be run with and without teleporting through wormholes.
- **Large Board with 4 players:** The basic game is played with two players on a small 5x5 board. The game can be played with four players (Red, Blue, Green and Yellow) on a larger 6x6 board. Having implemented this variation, the simulation can run with two different board sizes.

Advanced Features

If you have met the functional specification and completed the basic game with all the variations, then additional marks can be obtained by implementing advanced features.

Advanced features should only be attempted **once you have met the functional specification and completed the basic game with all the variations**. Implementing advanced features will require additional reading as the concepts and techniques will not necessarily be covered in lectures and labs. The accompanying course textbook will be a good starting point.

- **Game States.** Create a State Machine that starts a game in a **Ready** state, transitions to an **In Play** state when in play and transitions to a **Game Over** state when there is a winner. We require each state transition to be output to screen. In the **Game Over** state any additional attempt to play a piece by providing the game with additional dice rolls will result in outputting a 'Game Over' message. We require an implementation of a state machine using the State pattern as described in the lectures, labs or textbook, not using procedural logic (switch-case or if-then), to count as a successful variation.
- **Game Save and Replay – in memory.** A game is saved at the end of the game to an in-memory data store against some form of **Id** (a simple integer will suffice). The stored game can then be retrieved and the game replayed so that the board game designers can examine the output. To be a valid implementation you should store the game configuration (board size, number of players etc) and the sequence of dice rolls and replay the game using your game code. Capturing, storing and showing the textual output from the game is not a valid implementation of replay, but could get some credit. You must implement the replay feature so that any game that has been run in the session can be replayed, not just the most recent game.
- **Game Save and Replay -to file.** With an in-memory version of game save, saved games are lost when the program terminates. Extend the application to save games to a file and offer the user a choice of game to replay based on the contents of the file. You could save all games to one file or create many files. Use Spring Boot properties or profiles to switch between the in memory and file-based stores.

Architectural Specification

A development team in industry will have a coding standard and an architectural standard describing how software should be constructed. We will cover the architectural specification in the latter part of the module.

Your submitted code will need to fulfil the following coding standards

- Code to be written in Java 21 or higher. 25 is the current version at time of writing.
- Code to follow the Google Java coding guidelines available from <https://google.github.io/styleguide/javaguide.html>.
- Code should make use of design patterns to solve design problems. Your Q&A document should include an explanation of where and why design patterns have been used (naming the design pattern). Your code should demonstrate the use of as many patterns as possible as long as they are used appropriately. You can include design patterns not covered in the module material, but if so, you should provide a citation and reference to the source of the pattern in literature using the **Cite Them Right Harvard** referencing style.
- The application uses the Dependency Injection container from Spring Boot Framework.
- Is written in the Clean Architecture style discussed in the textbook and labs.
- Plays multiple games automatically.

The Clean Architecture style means that your submitted code will need to fulfil the following criteria:

- The domain logic (all the game mechanics) is separated from the technology (such as input from and output to the console and saving/retrieving files).
- Correctly places code into domain, usecase, and infrastructure packages according to their responsibilities.
- Package dependencies follow Clean Architecture principles (infrastructure depends on usecase, usecase depends on domain).
- Domain and use case code is testable with no direct dependencies on infrastructure code.
- The application is assembled using the Spring Dependency Injection container.

The Q&A Document (Report)

Please answer the questions in the Q&A template and submit this with your code. The questions are listed in the Appendix.

Your Q&A document must be written in **Markdown**, which is a lightweight markup language. Markdown is introduced in lectures and labs. The document file name therefore becomes QandA.md.

IDEs such as Intelli-J with a Markdown plugin will show a preview of the document.

Git Repository portals such as Github will automatically recognize and render a markdown file.

There is no word count limit- we are asking you to answer questions about your design and architecture, not to document your code, so as a guide 750-1250 words should be sufficient excluding tables and code examples.

- Use of Markdown formatting to create a visually pleasing document will be necessary for high marks.
- The use of the correct technical terms (see lecture notes and supporting material) will be essential for high marks.
- Higher marks can be obtained by correct use of UML diagrams. These can be provided using Mermaid (covered in lectures and labs).

To gather material to help you answer these questions, you will find it useful to keep a diary or journal whilst you are writing the assessment task code.

You might keep in your journal:

- A record of design decisions – what your decision was and its reasons, together with a note about the code change (git commit ids are useful here, you could also record notes in the commit message)
- Your own notes on the patterns and principles we discuss in the module, together with how you have applied them in your assessment code.
- Your thoughts and insights about your work at the end of each week.

How to show your application working

No GUI is required. Your application can simply write to the console, using (for example) the Java `System.out`. `printf` or `System.out`. format methods.

If you have implemented variations, then the game should be capable of being played with different variations without having to change or recompile code.

Your program should play a game demonstrating each variation automatically, running one game after another. It must be clear from the output which variations are being used and where a game starts and another finishes. There are many combinations of variations, but it is only necessary to demonstrate each variation once.

We would also suggest that you

- Show your 1 and 2 dice variations using random dice
- Develop and demonstrate the game variations using the different Exact End, Hit and Teleport rules using a fixed sequence of dice rolls.

If you implement save and replay, you will need to implement a command line menu to choose which saved game to run. This does not need to be a sophisticated user interface, just enough for the user to choose the game to replay.

Example output which meets the functional specifications is shown in the Appendix. You do not need to follow this output exactly, but your output should meet the functional specifications.

Support

There is planned time in labs to get support and formative feedback.

For additional support on the assessment, contact Steve Baker (steve.baker@mmu.ac.uk). Contact details and office hours are on the module page in Moodle.

Viva

After the submission deadline, you may be invited to attend a viva to discuss and demonstrate your understanding of the code you have submitted. Some the reasons this may occur following your submission are:

- Lack of congruity between code and Q&A
- Lack of understanding shown in Q&A
- Suspected use of AI in generating sections of text or code
- Code submissions that greatly deviate from the provided material
- Similarity to other student's submissions or other reasons to believe academic misconduct has taken place

If explanations for sections of code are not satisfactory, your mark may be downgraded or capped. Please also note that if you are due to undergo a requested viva as part of your assessment, your mark can and will be withheld until this has been completed.

Suggested Approach and Schedule

The module covers design first and then architecture from around week 7/8. Therefore, you should concentrate on the functional specification and the application of design patterns first and then refactor your code to the end-state architecture after we have covered the architectural specifications.

Do not leave starting this assignment to the end of the module – good design takes time and many iterations, and you will not be making good use of lab sessions for formative feedback.

Ensure you have a working development environment as soon as possible – we set this up in the first week lab.

The essential skills of software design are identifying and creating classes and applying design patterns.

- Classes should be written and laid out using the coding and layout conventions discussed in lectures and labs.
- Code should use object-oriented techniques such as **interface implementation, inheritance, polymorphism, and encapsulation**.
- Classes should conform to the **SOLID (single responsibility, open/closed, Liskov substitution, interface segregation and dependency inversion)** principles.
- Standard **design patterns** used where appropriate.
- The implementation makes minimal use of if/else or switch statements, preferring instead to use abstractions and strategies.

A suggested schedule is included in the Appendix.

Hints and Tips for designing and writing the Functional Code

Review the coding conventions covered in week 1. Ensure you are writing well formatted code that conforms to the coding conventions.

There is no “right” design. We are looking for sensible use of patterns (too much pattern usage better than not enough in this case) and a good assignment of functionality to well named classes.

It is unlikely you will get the design right first time – an iterative approach will be more successful: write a version, take a critical look (analyse) your design using the knowledge gained from the module, and reflect on where it can be improved, decide what you are going to change and produce another version.

Use of Git is highly recommended. Create a Git branch for each design change – not all design experiments are successful, and being able to revert to your previous version can save a lot of time.

You are going to need both a random number generator for simulating the dice, but for testing and development purposes this will become very frustrating (no pun intended). Therefore, we would recommend that a random dice roll will need to be swappable with a dice role generator that creates a fixed sequence of rolls, so that you can create a fixed sequence of dice rolls for testing purposes. The examples in the Appendix show relevant dice sequences.

Unit testing using JUnit was introduced in Advanced Programming. Although not required for this assessment, you may find it helpful to write unit tests for your individual classes. If you do produce unit tests **include them in your submission.**

Ensure you have a code version that compiles and executes that you could hand in at the end every week.

Hints and Tips for meeting the Architecture Specification

Using the website <https://start.spring.io> create a starter project using the following settings

- Project: Maven
- Language: Java
- Sprint Boot: Choose latest stable (at time of writing this was 3.5.5)
- Project Metadata – Group: uk.ac.mmu
- Project Metadata – Artifact: game
- No dependencies are required.

Generate a project which will provide the basis for your solution. We will also provide an example in the labs.

Clean Architecture (Ports and Adaptors)

An example of a clean architecture using ports and adaptors will be provided in labs.

Use Cases

You will need to implement the following use cases for a complete architecture.

Play – given a configuration (board size, number of players, variations) play the game to completion and save the game for later replay to a data store.

Replay – given the id of a game, retrieve it from the data store and replay it.

Saving Games

In Clean Architecture the games are saved and retrieved via Persistence Ports and using Persistence Adaptors (The Repository Pattern). You can write a persistence adaptor that saves games to memory or to a file. Ideally you will write both and demonstrate that they can be swapped by changing the DI Container configuration.

Using JSON serialisation

If you decide to write your game data to a file using JSON serialisation, one standard library is Jackson.

The simplest method of bringing in Jackson is to use spring-boot-starter-json.

Change the dependency in the pom.xml generated by start.spring.io from **spring-boot-starter** to **spring-boot-starter-json**

```
<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-json</artifactId>

</dependency>
```

If you change the pom.xml file manually in the texteditor, you need to synchronize the changes.

1. In the Maven tool window, right-click a linked project.
2. From the context menu, select Sync Project the Sync project icon.

See <https://www.jetbrains.com/help/idea/delegate-build-and-run-actions-to-maven.html>

Follow instructions in <https://github.com/FasterXML/jackson-databind> for how to serialize and deserialize objects to JSON.

Locating your file

You can store your game files anywhere, but usually these kinds of files would be stored in a user's private area. Unfortunately, where this is depends on operating system – Windows, Linux and MacOS will all use a different location on disk for user files.

Java has a useful properties collection in the System class that abstracts away some of the operating system specifics, specifically the "user.home" property which provides the user's home directory as a string in an OS independent way.

```
String home = System.getProperty("user.home");
```

See <https://docs.oracle.com/javase/tutorial/essential/environment/sysprop.html>

Scheme

Criteria and % of Overall Mark	Clear Fail 0-29%	Marginal Fail 30-39%	Pass 40-49%	2:2 50-59%	2:1 60-69%	1 70-100%
Functionality provided by the submitted code (40 marks) <i>Code submitted without an accompanying Q&A.md document will NOT be marked.</i>	Code does not compile or does not run, or minimal functionality.	Basic game attempted with minimal success.	Basic game attempted with some success; some variations attempted.	Basic game functions successfully, some variations implemented.	Basic game functions successfully, all variations implemented.	Basic game functions successfully and all variations implemented. Advanced features attempted or implemented. Product has high-quality input validation and high-quality outputs.
Quality of Submitted code (40 marks) <i>Code submitted without an accompanying Q&A.md document will NOT be marked.</i>	Code does not compile or does not run or minimal functionality or architecture	Poor quality code that has no or little object-orientation, does not follow SOLID principles. Code does not conform to the architectural specification.	Code has some object-orientation and follows some SOLID principles. Code conforms to some elements of the architectural specification.	Code is generally object orientated and generally follows SOLID principles. Code generally conforms to the architectural specification.	Code is completely object-oriented, follows SOLID principles making use of design patterns where appropriate. Code conforms to the architectural specification.	Code is completely object-oriented, expertly follows SOLID principles and makes extensive use of design patterns where appropriate. Code conforms to the architectural specifications in an exemplary manner.
Quality of Q&A Document. (20 marks) <i>A Q&A document submitted without accompanying code will NOT be marked.</i>	No Q&A or only basic Q&A document provided.	Product design and architecture has not been evaluated. Few, if any, connections are made between the product and the patterns and principles discussed in the module. No evidence of personal insight into design and architecture having been gained.	There is limited evaluation of product design and architecture. Few connections are made between the product and the patterns and principles discussed in the module. Limited evidence of personal insight into design and architecture having been gained.	There is some evaluation of product design and architecture. Some connections are made between the product and the patterns and principles discussed in the module. Evidence of personal insight into design and architecture having been gained.	There is a good evaluation of product design and architecture. Clear connections are made between the product and the patterns and principles discussed in the module. Evidence of multiple personal insights into design and architecture having been gained.	There is an expert and critical evaluation of product design and architecture. Superior connections are made between the product and the patterns and principles discussed in the module. Evidence of multiple personal insights into design and architecture having been gained which are articulated as if to a professional audience.

Appendix A - Game Rules

21	22	23	24	25 
20	19	18	17	16
11	12	13	14	15
10	9	8	7	6
1 	2	3	4	5

1

Basic Game Rules

This is a game for two players. **Red** and **Blue**.

- The basic game is played on a 5x5 board with position numbers as shown.
- Each player starts with a player piece in their **Home** position. The Red **Home** position is 1 and **End** position is 25. The Blue **Home** position is 25 and **End** position is 1.
- To win the game, a player must get their player piece to their **End** position first.

- Players take turns to roll 2 x 6-sided dice.
- Players follow a **boustrophedon** (ox-plow) track from their **Home** position to their **End** position.
- The **Red** player moves **Home** (Position 1), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, **End** (Position 25)
- The **Blue** player moves **Home** (Position 25), 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, **End** (Position 1)
- If a player piece lands on a position occupied by another player piece, then this is called a **Hit**. In the basic game Hits are ignored which allows for multiple players to occupy the same position.
- The game continues until either Red or Blue have reached their **End** position. This could be done by throwing a dice roll that lands the player on or beyond the **End** position. Throwing a dice roll that would move the player beyond the **End** position is called an **Overshoot**. In the basic game a player can win by landing on the End position exactly or overshooting.

Variation Rules

Exact End: Player must land exactly on the END position to win else the player bounces back.

The player must land exactly on the END position to win. If the player overshoots, the player 'bounces' off the END position using the same boustrophedon track but in reverse.

For example, in an 5x5 board.

If **Red** was on position 20 and rolls a 12 then Red overshoots the **End** and moves from position 20 to position 18. Red would need to roll 7 on the next turn to win.

If **Blue** was on position 6 and rolls a 7 then Blue overshoots the **End** and moves from position 6 to position 3. Blue would need to roll 2 on the next turn to win.

Hit: Player's turn is forfeit if the player would HIT another player

If a player move would occupy the same position as another player, this is a HIT and the moving player stays where they are – the move is forfeit (although it still counts as a turn).

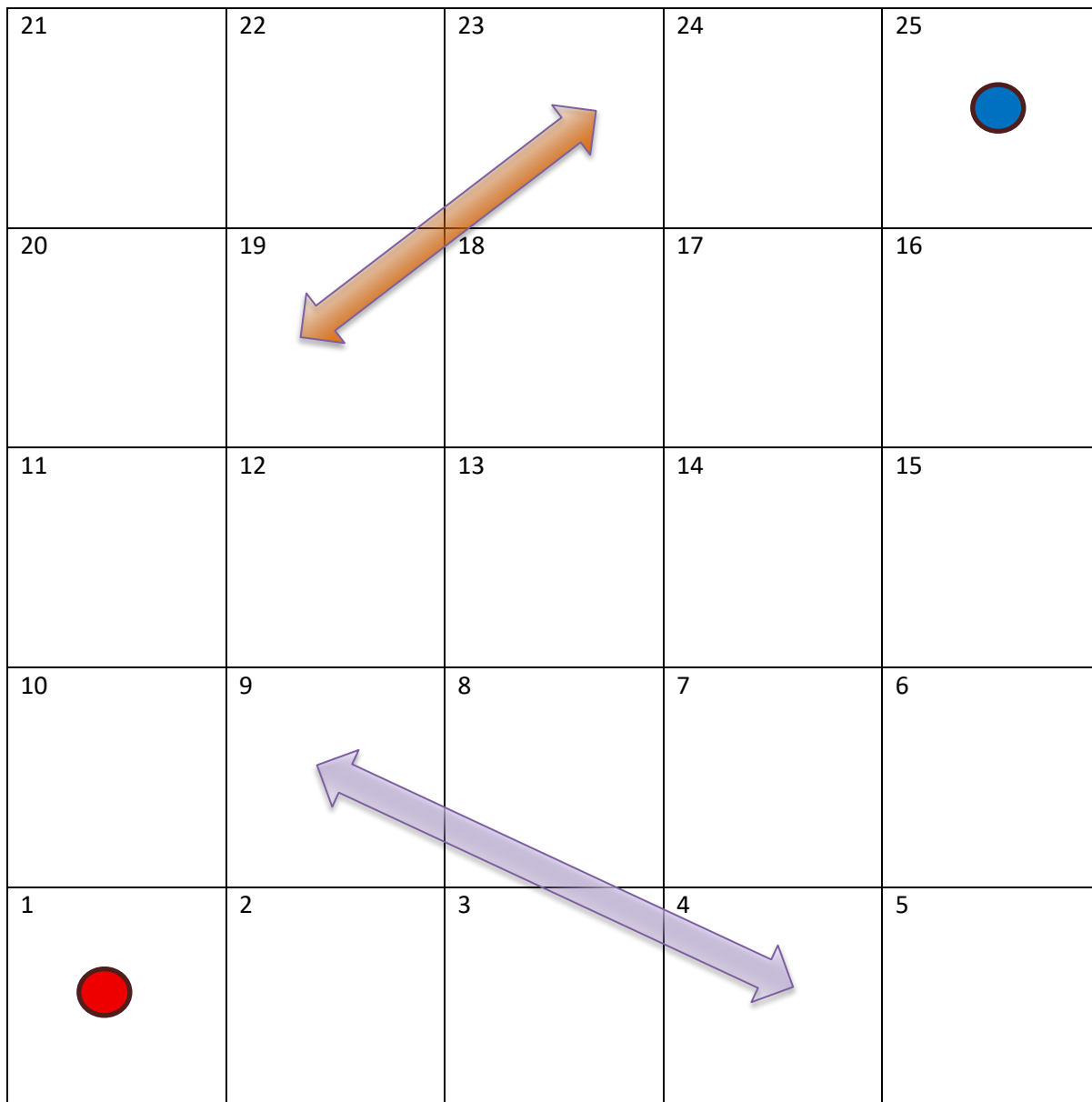
For example, on a 5x5 board

Blue is on position 19.

Red is on position 13.

If **Red** rolls 6, then it would move from 13 to 19, but this would be a HIT on **Blue** at position 19, so **Red** moves back from 19 to 13.

Teleport: Player is teleported to the other end of the wormhole



The board can be configured with positions that are connected via ‘wormholes’. In the example above there are two wormholes between positions 4 and 9 and between positions 19 and 23.

If a player lands on either end of a ‘wormhole’ they are ‘teleported’ to the other end (the player enters at one end and exits at the other).

A wormhole can either advance a player or set them back depending on the position of the wormhole. Teleporting may be advantageous or disadvantageous to the player depending on their starting position and direction of travel.

Using the example above, a player could:

- land on 4 and be teleported to 9.
- land on 9 and be teleported to 4.
- land on 23 and be teleported to 19.
- land on 19 and be teleported to 23.

There are some rules regarding the placement of wormholes

- Wormholes must go between two distinct positions, i.e. a wormhole cannot have an entrance and exit at the same position.
- Wormholes cannot enter or exit on any player's **Home** or **End** positions. Which positions are regarded as player Home or End positions depends on the size of the board and the number of players. In the example shown wormholes cannot have positions 1 or 25 as an entrance or exit.

Your implementation should validate any wormholes configured for a board meet these rules.

If the game is configured to have wormholes, your game should implement a **teleport** rule that either ignores wormholes (i.e. landing on one of the ends of a wormhole has no effect) or causes the player to be teleported (i.e. landing on one of the ends of a wormhole causes the player to be teleported to the other end of the wormhole).

Large Four Player Board

A variation is to play with four players **Red**, **Blue**, **Green** and **Yellow** on a larger 6x6 board.

36 	35	34	33	32	31 
25	26	27	28	29	30
24	23	22	21	20	19
13	14	15	16	17	18
12	11	10	9	8	7
1 	2	3	4	5	6 

Red moves **Home** (Position 1), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, **End** (Position 36)

Blue moves **Home** (Position 31), 32, 33, 34, 35, 36, 25, 26, 27, 28, 29, 30, 19, 20, 21, 22, 23, 24, 13, 14, 15, 16, 17, 18, 7, 8, 9, 10, 11, 12, 1, 2, 3, 4, 5, **End** (Position 6)

Yellow moves **Home** (Position 36), 35, 34, 33, 32, 31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, **End** (Position 1)

Green moves **Home** (Position 6), 5, 4, 3, 2, 1, 12, 11, 10, 9, 8, 7, 18, 17, 16, 15, 14, 13, 24, 23, 22, 21, 20, 19, 30, 29, 28, 27, 26, 25, 36, 35, 34, 33, 32, **End** (Position 31).

These are some example games that show how the basic game works and has the required output to meet the functional requirements.

Basic game: Red wins in 2 turns (3 in total).

Board: Small, Players: 2, Exact End: Player can land on or overshoot the END position to win, Hit: HITS are ignored, multiple players can occupy the same position, Teleport: Wormholes are ignored, Wormholes: None

Fixed sequence of dice rolls {12,10,12}

Game

Board: rows=5 columns=5

Red Home (Position 1), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, End (Position 25)

Blue Home (Position 25), 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, End (Position 1)

Game State: Ready → InPlay

Red turn 1 rolls 12

Red moves from Home (Position 1) to 13

Blue turn 1 rolls 10

Blue moves from Home (Position 25) to 15

Red turn 2 rolls 12

Red moves from 13 to End (Position 25)

Red wins in 2 turns.

Total turns: 3.

Game State: InPlay → GameOver

Dice rolls: {12, 10, 12}

Game Id: 1 saved.

Basic game: Blue wins in 2 turns (4 in total).

Board: Small, Players: 2, Exact End: Player can land on or overshoot the END position to win, Hit: HITS are ignored, multiple players can occupy the same position, Teleport: Wormholes are ignored, Wormholes: None

Fixed sequence of dice rolls {5,12,2,12}

Game

Board: rows=5 columns=5

Red Home (Position 1), 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, End (Position 25)

Blue Home (Position 25), 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, End (Position 1)

Game State: Ready → InPlay

Red turn 1 rolls 5

Red moves from Home (Position 1) to 6

Blue turn 1 rolls 12

Blue moves from Home (Position 25) to 13

Red turn 2 rolls 2

Red moves from 6 to 8

Blue turn 2 rolls 12

Blue moves from 13 to End (Position 1)

Blue wins in 2 turns.

Total turns: 4.

Game State: InPlay → GameOver

Dice rolls: {5, 12, 2, 12}

Game Id: 2 saved.

Appendix B – Q&A Document Template

We are asking you to critically evaluate your architecture and design by providing a document (written in Markdown) that answers the following questions.

As this is an evaluation, there are no 'right' or 'wrong' answers, we are looking for an honest evaluation of your software product. We would suggest 750-1250 words (excluding tables and code examples) but this is not a limit.

1. Which design and architecture patterns did you use and why? (you should include examples from your software product to illustrate your use).
2. Which element(s) for your design and architecture are you most pleased with and why?
3. Which element(s) for your design and architecture are you least pleased with and why? What could you do to improve those elements if you had more time?
4. Describe how your design evolved over time.
5. How do you think your code follows the principles we have discussed in the module? (use examples from your software product to illustrate where you think principles have or have not been followed well).
6. Can you identify insights you have personally gained about software design and architecture from this module? How did you arrive at your insights (you might need to go back through your Git history, lab exercises or notes looking for 'lightbulb moments').

Appendix C – Suggested Schedule

Week	Schedule
1	Have your development environment setup. We strongly advise the use of Git to manage your work, so get a Github account and practice basic Git operations.
2	Start by quickly coding the basic game. At this stage you are not designing, you are just writing some code (you could try and do everything in the single main method if you wish) to explore and understand the problem. Check your understanding of the game against the specification.
3	Refactor your basic game code into classes. A superior design will require many classes. Take time to identify the classes and assign responsibilities to them using the principles we have discussed in the material.
4	Code some variations and feature. Having coded an object-oriented version of the basic game first, you will have a good understanding of the game works and therefore will be in a better position for tackling the variations requested by the designers. We would suggest taking on each requested variation/feature one at a time. Trying to write an initial version that implements everything will be overwhelming, and taking an iterative approach means that you will always have a working version to submit.
5	Continue with the variations. Refactor the code to use patterns where possible. Attempt advanced features if you have got all the variations done.
6,7	Review and get feedback on your design at the lab sessions. Refactor the code based on feedback.
8	By 8 your game code should be complete and tested.
9	Ensure you have worked through the Spring Boot labs, this will be important for the final part of the assessment
10	Setup your Clean Architecture framework ready to receive your game code
11	Move your game code into your Clean Architecture implementation. Review and get feedback on your architecture at the lab sessions. Refactor the code based on

	feedback
12+	Ideally you have used the time during the semester to build your assessment code up by week, so during assessment weeks you can concentrate on testing, fine tuning the design and the Q&A document

Appendix D: Assessment Mapping to KSBs

Software Design and Architecture	
Core Knowledge	Whether requires evidence for Professional Discussion
A range of digital technology solution development techniques and tools. (K5)	
The approaches and techniques used throughout the digital and technology solution lifecycle and their applicability to an organisation's standards and pre-existing tools. (K6)	Y
Techniques of robust research and evaluation for the justification of digital and technology solutions. (K18)	
Core Skills	
Analyse a business problem to identify the role of digital and technology solutions. (S1)	
Initiate, design, code, test and debug a software component for a digital and technology solution. (S4)	Y
Apply relevant standard processes, methods, techniques and tools. For example, ISO Standards, Waterfall, Agile in a digital and technology solution project. (S5)	
Software Engineering Knowledge	
How to operate at all stages of the software development life cycle and how each stage is applied in a range of contexts. For example, requirements analysis, design, development, testing, implementation. (K21-SEK1)	Y
Principles of a range of development techniques, for each stage of the software development cycle that produce artefacts and the contexts in which they can be applied. For example UML, unit testing, programming, debugging, frameworks, architectures. (K22-SEK2)	Y
How to interpret and implement a design, compliant with functional, non-functional and security requirements including principles and approaches to addressing legacy software development issues from a technical and socio-technical perspective. For example architectures, languages, operating systems, hardware, business change. (K24-SEK4)	Y
The factors affecting product quality and approaches for how to control them throughout the development process. For example security, code quality, coding standards. (K25)	
How to select and apply a range of software tools used in Software Engineering. (K26)	
Approaches to the interpretation and use of artefacts. For example UML, unit tests, architecture. (K27)	
Software Engineering Skills	

Identify and define software engineering problems that are non-routine and incompletely specified. (S16)	
Provide recommendations as to the most appropriate software engineering solution. (S17)	
Use appropriate analysis methods, approaches and techniques in software engineering projects to deliver an outcome that meets requirements. (S18)	
Implement software engineering projects using appropriate software engineering methods, approaches and techniques. (S19)	
Determine, refine, adapt and use appropriate software engineering methods, approaches and techniques to evaluate software engineering project outcomes. (S21-SES7)	Y